

Marcador WIG 4DX

A 4DX execution scoreboard for GBM Nicaragua

Company WIG: \$1M net profit after taxes (NAT) in 2027 → \$2M in 2028 → \$2.4M in 2029, supported by 12 WIGs with weekly & monthly lead measures.

Project overview · June 2026

Maintainer: Plans & Controls (Danilo Chamorro) · Owners: 12 functional leads

What this project is

This is the scoreboard system behind GBM Nicaragua's **4DX** (4 Disciplines of Execution) program. The whole organization is steering toward one Wildly Important Goal — the **company WIG**: reach **\$1M of net profit after taxes (NAT) in 2027**, then \$2M in 2028 and ~\$2.4M in 2029 (base year 2026).

That single number is too far away to manage week to week, so it is broken down into **12 supporting WIGs**. Each WIG has one *lag* measure (the result you want) and several *lead* measures (the weekly or monthly behaviors the team can actually influence). Owners report on their lead measures in the Monday cadence; the lag only moves if the leads do.

The product tension it solves

There is a real conflict between two things the program needs at once: owners must be able to update their numbers with **zero friction** (so the data is current), and leadership needs a **glanceable scoreboard** readable from across a room in five seconds (so the cadence has teeth). A spreadsheet is great for the first and poor for the second; a BI tool is the reverse and adds adoption cost. This project keeps **Excel as the input surface** and renders a **self-contained TV scoreboard** on top of it — owners keep typing in cells they already understand, and the room sees a clean, rotating board.

How it works

Structure lives in code; data lives in cells. A Python script generates the workbook's fixed structure, owners fill the blue input cells, and a single HTML file parses the workbook by fixed cell positions and turns it into rotating slides.

Stage	What happens
1 · Excel	Owners type weekly/monthly results into blue input cells (shared copy on the PGX server).
2 · build_wig.py	Generates the workbook <i>structure</i> — the 12 WIG tabs, Dashboard, year pages, Tareas. Single source of truth.
3 · recalc	LibreOffice headless computes the formulas and caches the values (mandatory — the reader only sees cached values).
4 · marcador.html	Parses the .xlsx by fixed positions (SheetJS, embedded) and builds the slides; offline-first, no backend.
5 · Display	Office TV reads it over the LAN in kiosk Chrome; a login-gated Azure copy serves remote viewers.

Slide order on the TV: Company dashboard → 2027 → 2028 → 2029 → the 12 WIGs. Touching a lead opens its detail (history + supporting tasks).

The 12 supporting WIGs

Each rolls up to the company NAT goal. Targets in parentheses; “X/Y” means the threshold is still being defined by the owner.

#	WIG (lag measure)	Target
1	Grow the Smart User fleet	+500 devices placed by Dec 2026
2	Advisory Monthly Value	\$X of advisory MV by Q1 2027
3	Large deals (>\$30K GP)	10 deals recognized in 2027
4	On-time delivery	100% on-time placements/deliveries
5	Datacenter (San Marcos) MV	+\$20,000 incremental MV in 2027
6	Zero expired contracts	0 expired, 16 of next 18 months
7	Digital Solutions MV	+\$30K/month by May 2027
8	Consolidated gross profit	16% GP sustained in 2027
9	Customer satisfaction (CSAT)	X→Y, sustained 4 months
10	Operating leverage	Local cost flat as revenue grows (≤9%)
11	Recover Digital Solutions margin	Negative lines → GP ≥ \$0
12	Billable engineer utilization	42% → 48%+ sustained

The stack

Layer	Technology
Data / structure	Python 3 + openpyxl (build_wig.py generates the workbook; migrate_data.py moves owner data forward), LibreOffice headless for formula recalculation.
Display	web/marcador.html — one self-contained file with SheetJS embedded. Vanilla JS, no framework, offline-first. Light/dark theme, rotating slides, touch drill-down to each lead's history and supporting tasks.
Distribution — LAN	PGX server (Lenovo, DGX OS): Samba shares the workbook, a systemd timer validates & publishes it, nginx serves the board. Office TV runs kiosk Chrome — no login on the LAN.
Distribution — remote	Azure Static Web Apps (Standard tier) with Entra ID single-tenant login — only @caobagroup.com accounts. The authenticated copy for remote viewers.
CI / publish	GitHub Actions (azure-static-web-apps.yml) deploys web/ on push to main; make_publicar validates the contract + recalc before publishing.
Tests	Contract tests (tests/test_contract.py) enforce that the Excel layout and the HTML parser stay in sync.
Explorations (not live)	Airtable “Option A” as a future input layer; TRMNL e-ink desktop cards rendered per person (trmnl_render.py).

Next steps

Operational rollout (near-term)

Getting the board live and the cadence running:

- **Provision the PGX server** — run `deploy/setup-wig.sh` (Samba + nginx + publish timer), copy `marcador.html` and the workbook, map the share on owner machines, and set the office TV to kiosk Chrome.
- **Configure remote login** — finish the Azure Entra ID setup per `docs/AZURE_LOGIN.md` (register the app, set the tenant ID and secret) so the remote copy is gated to company accounts.
- **Establish the Monday ritual** — owners enter their numbers in the shared workbook; the coordinator runs `make publicar DATOS=...` to validate and publish. Lock in this weekly cadence.

Roadmap (deferred — revisit as the cadence matures)

- **SharePoint co-editing** — replace save-and-close Samba sharing if simultaneous edits become a pain point.
- **Move the data source off Excel** — Airtable “Option A” is prototyped (`build_airtable.py`); the `sync_from_airtable.py` adapter is not yet written.
- **History & trends** — keep period-over-period data to show trajectories, not just the current week.
- **Alerting** — notify owners when a WIG slips into Riesgo/Atrasado.
- **TRMNL e-ink cards** — always-on personal desktop scoreboards (mockups and renderer exist; not connected to live data).
- **Multi-country rollout** — generalize beyond Nicaragua.

Technical appendix

For maintainers. Full detail lives in `CLAUDE.md`, `docs/PRD.md` and `docs/DESIGN.md`.

The Excel ↔ parser contract

`parseWorkbook()` in `marcador.html` reads **fixed cell positions** — there are no headers or named ranges to rely on. The hard rules:

- Data starts at **row 11** (DATA0); each WIG has at most **8 leads** (MAX_LEADS).
- Status semaphore (exact Spanish text): **En meta** ≥95% · **Riesgo** 80–95% · **Atrasado** <80%.
- **Three-files-in-one-commit rule:** any structural change must touch `build_wig.py`, the parser in `marcador.html`, and `tests/test_contract.py` together — never hand-edit the `.xlsx` for structure.

Key commands

Command	What it does
<code>make build</code>	Generate the workbook structure + recalc.
<code>make demo</code>	Workbook with dummy data + a standalone demo board.
<code>make test</code>	Run the contract tests.
<code>make migrate OLD=...</code>	Carry owner data forward into a rebuilt workbook.
<code>make publicar DATOS=...</code>	Validate (contract + recalc) and publish to Azure.
<code>make pdf</code>	Regenerate this overview PDF.

Repository layout

Path	Contents
<code>excel/</code>	Workbook generator, data migrator, demo filler, Airtable seeder.
<code>scripts/</code>	recalc (mandatory), demo embedder, TRMNL card renderer.
<code>web/</code>	<code>marcador.html</code> (the TV board), published workbook, Azure config.
<code>deploy/</code>	PGX provisioning script (Samba + nginx + timer).
<code>docs/</code>	PRD, DESIGN, Airtable notes, Azure login & rollout runbooks.
<code>tests/</code>	Contract tests (Excel↔parser) + TRMNL render checks.
<code>dist/</code>	Build artifacts (gitignored) — including this PDF.